Дискретный анализ

Вайб-лекция 6: суффиксные деревья

Андрей Калинин

Дискретный анализ, 2026/27

21 октября 2026

Где остановились и куда идём

Прошли 14.10	Доскажем сегодня	Новая тема
фильтр Блума; ошибка CPython; Z, КМП, Бойер–Мур, AG	Two-Way и как CPython починили поиск; SIMD-фильтр коротко; замер; Ахо–Корасик	один текст, много запросов: задача MUMmer и суффиксное дерево

Пример тот же: текст $T = \text{abaabaabacb}$, образец $P = \text{abaab}$, вхождения 0 и 3.

Вопрос прошлого раза остаётся: **фильтр + точная проверка**, и как не сравнивать дважды. Сегодня добавится третий: **что сохранить о тексте**, если его читают тысячи раз.

План дня

1. Хвост лекции 5: Two-Way, адаптивный CPython, SIMD-фильтр, сравнение девяти реализаций, Ахо–Корасик.
2. Задача из биологии: сравнить два генома. Как это делает MUMmer и что ему нужно от структуры данных.
3. Суффиксное дерево: определение, размер, поиск за $O(m + \mathrm{occ})$.
4. Алгоритм Укконена: как построить дерево за $O(n)$, на строке `abaab$`.
5. Что дерево умеет: повторы, общая подстрока двух текстов, MUM.

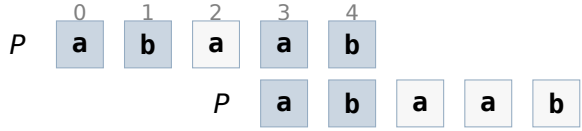
Следующая лекция: суффиксный массив и LCP — то же дерево в двух массивах, и почему MUMmer4 перешёл на них.

Two-Way

Линейный поиск без таблиц, и как CPython починил `bytes.find`

Период строки

p — **период** строки P, если $P[i] = P[i+p]$ при всех $0 \le i < m - p$: строка совпадает сама с собой, сдвинутой на p. Наименьший период обозначаем $p(P)$.

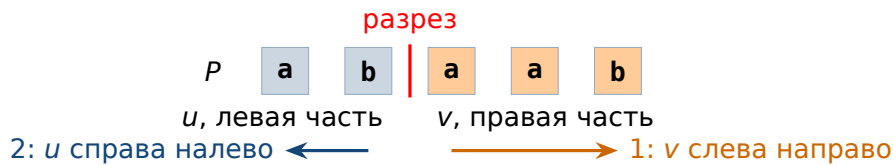


У abaab период 3: сдвинутая копия совпадает на ab. Период и граница — одно и то же знание: $p(P) = m - \pi[m-1]$ (граница ab длины 2, $5 - 2 = 3$).

Примеры: ABABAB — период 2 (повтор может быть неполным: ABABA тоже 2); GCAGAGAG — период 7 (совпадает только последняя G); строка без границы — период m.

Two-Way: разрезать образец и сравнивать в две стороны

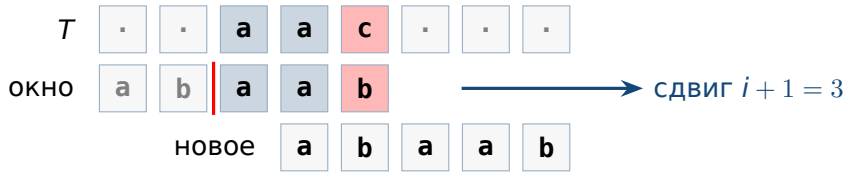
Алгоритм Крочемора и Перрена (1991): линейное время без таблиц. Об образце хранятся **два числа**: где резать и период.



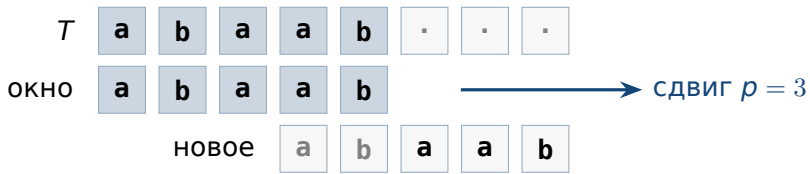
Сначала проверяется правая часть, от разреза вправо. Только если она совпала целиком — левая, от разреза влево. Для abaab: разрез после второй буквы, $u = \text{ab}$, $v = \text{aab}$, период 3.

Два правила сдвига — на картинках

Правило 1. Различие в правой части, когда от разреза совпало i букв: сдвиг на $i + 1$.

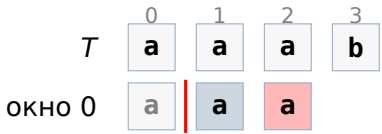


Правило 2. Правая часть совпала, дальше различие в левой или полное совпадение: сдвиг на период p; первые $m - p$ букв нового окна уже известны (серые).



Вопрос залу

Где резать? Образец aab, текст aaab. Попробуем разрез после первой буквы.



Правая часть ab: $T[1] =$ a совпала, $T[2] =$ a $\neq$ b — различие после $i = 1$ буквы. Правило 1: сдвиг на 2. Что произошло с вхождением в позиции 1?

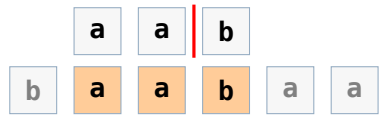
Ответ: такой разрез теряет вхождение; нужен разрез без «эха»

Разрез a | ab: эхо a по обе стороны



локальный период 1

Разрез aa | b: кратчайшее эхо — baa, вылезает за край



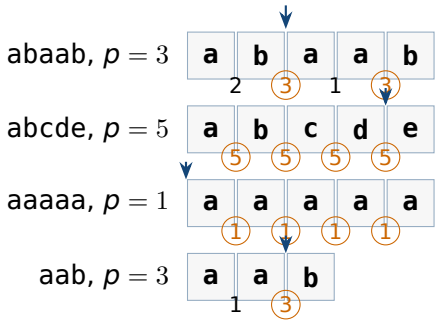
локальный период 3

Локальный период разреза — длина кратчайшего «эха»: слова, которое стоит и слева, и справа от разреза (может выступать за края). Он не больше периода всей строки. Разрез **критический**, если они равны. Сдвиг d по правилу 1 означал бы эхо длины d через разрез — при критическом разрезе оно не короче p.

Crochemore, Perrin, *Two-way string-matching*, J. ACM 38(3), 1991, лемма 3; теорема о критической факторизации — Cesari, Vincent 1978.

Критических разрезов может быть несколько

Число у разреза — локальный период; в кружке — критический (равен p); стрелка — выбор алгоритма.

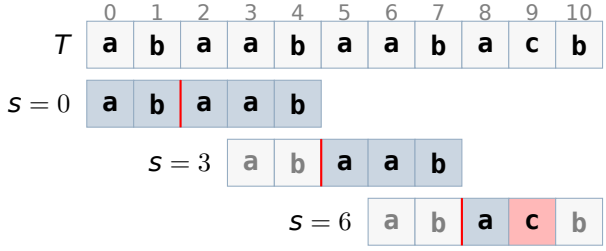


- abaab: критичны разрезы 2 и 4; алгоритм берёт 2.
- Разные буквы: эхо не короче всей строки — критичен любой разрез; алгоритм берёт последний.
- Одна буква: эхо a везде, $p = 1$, критичен любой разрез; алгоритм берёт нулевой: левая часть пуста.
- aab: критичен только разрез 2 — тот, что нашёл вхождение.

Посчитано demo/twoway_cuts.py. Критический разрез всегда есть среди $0..p-1$ (Cesari, Vincent 1978); алгоритм находит его за $O(m)$ как более поздний из двух максимальных суффиксов.

Трасса Two-Way: abaab в abaabaabacb

$u = \mathrm{ab}$, $v = \mathrm{aab}$, $p = 3$, образец периодический ($u = P[3..5)$). Синее — сравнили; серое — известно без сравнения; красное — различие.



s	memory	v слева направо	u справа налево	сдвиг
0	0	aab = $T[2..5)$	ba — **вхождение 0**	$p = 3$, memory $= 2$
3	2	aab = $T[5..8)$	известны — **вхождение 3**	$p = 3$, memory $= 2$
6	2	$T[9] = \mathrm{c} \neq \mathrm{a}$: $i = 1$	—	$i + 1 = 2$, memory $= 0$

/20261014/demo/twoway_trace.py. Crochemore, Perrin 1991, теорема 3.

Настоящий код: CPython проверяет периодичность

```
313     p->cut = STRINGLIB(_factorize)(needle,
        len_needle, &(p->period));
314     assert(p->period + p->cut <= len_needle);
315     p->is_periodic = (0 == memcmp(needle,
316                                   needle +
        p->period,
317                                   p->cut *
        STRINGLIB_SIZEOF_CHAR));
318     if (p->is_periodic) {
319         assert(p->cut <= len_needle/2);
320         assert(p->cut < p->period);
321     }
322     else {
323         // A lower bound on the period
324         p->period = Py_MAX(p->cut, len_needle -
        p->cut) + 1;
325     }
```

- Строка 313: `_factorize` — два максимальных суффикса, более поздний разрез; p здесь структура подготовки, не образец.
- Строки 315–317: периодичен ли образец — совпадает ли u с образцом, сдвинутым на период. Для abaab: ab = ab, да.
- Строки 322–324: иначе период неизвестен, берётся нижняя оценка $\max(\ell, m - \ell) + 1$ — ею и сдвигаемся.

python/cpython 3.10.0 · PSF Objects/stringlib/fastsearch.h:313–325

Вопрос залу

Two-Way линеен и не требует памяти. Почему бы не включить его всегда?

Подсказка: вспомните формулу окупаемости фильтра. Что здесь «фильтр», а что «точная проверка»?

Ответ: подготовка Two-Way стоит $O(m)$

Старый цикл — дешёвый фильтр: на обычных данных почти ничего не читает. Two-Way сначала платит $O(m)$ за разрез; при коротком тексте это дороже поиска. Нужна **политика**: когда платить.

```
554     if (mode != FAST_RSEARCH) {
555         if (m >= 100 && w >= 2000 && w / m >= 5) {
556             /* For larger problems where the needle isn't a
        huge
557                percentage of the size of the haystack, the
        relatively
558                expensive O(m) startup cost of the two-way
        algorithm
559                will surely pay off. */
560             if (mode == FAST_SEARCH) {
561                 return STRINGLIB(_two_way_find)(s, n, p, m);
562             }
563             else {
564                 return STRINGLIB(_two_way_count)(s, n, p,
        m, maxcount);
565             }
```

- Строка 554: длинный образец, много позиций ($w = n - m$), текст заметно длиннее — сразу Two-Way. Пороги 100, 2000, 5 подобраны по замерам.

python/cpython 3.10.0 · PSF fastsearch.h:554–565

Вторая ветка: начать дёшево и считать потраченное

```
609                 /* miss: check if next
    character is part of pattern */
610                 if (!STRINGLIB_BLOOM(mask,
    ss[i+1])) {
611                     i = i + m;
612                 }
613                 else {
614                     i = i + skip;
615                 }
616                 hits += j + 1;
617                 if (hits >= m / 4 && i < w -
    1000) {
618                     /* We've done O(m)
    fruitless comparisons
619                        anyway, so spend the
    O(m) cost on the
620                        setup for the two-way
    algorithm. */
```

python/cpython 3.10.0 · PSF fastsearch.h:609–620

- Строка 616: `hits` копит число *бесплодных* сравнений.
- Строка 617: набралось $m/4$ — работа, сравнимая с подготовкой, уже потрачена; переключаемся на Two-Way, если впереди достаточно текста.
- Фильтр не выключили — за ним следят: формула окупаемости стала счётчиком.

perf: исчезла работа, а не промахи предсказателя

На один вызов, приближённо	3.9.0	3.10.0
Инструкции	192 979 197	1 774 559
Ветвления	64 208 029	418 102
Такты	62 789 048	413 599

Промахи предсказателя у обоих — не более $0{,}01\%$. Ветвления были *предсказуемыми*: процессор исполнял их быстро, просто их было в 150 раз больше.

В лекции 1 мы видели обратное: столько же работы, но промахи. Счётчики различают эти два случая; секундомер — нет.

Наш запуск, CPU 1, `perf stat -r5`: cycles, instructions, branches, branch-misses; demo/perf-cpython39.txt, perf-cpython310.txt; numbers.md, N02.

Что сделал программист

1. Получил воспроизводитель; сохранил тот же ответ.
2. Нашёл ветку, которая делает лишнюю работу, — и убедился, что фильтр ни при чём.
3. Добавил алгоритм с гарантией худшего случая.
4. Измерил не только плохой вход, но и обычные — и нашёл регрессии.
5. Выбрал, *когда* за гарантию платить: пороги и счётчик.

Та же дилемма повторится сейчас с очень дешёвым фильтром — SIMD.

SIMD-фильтр

Самый дешёвый фильтр: 32 кандидата за одну инструкцию

Можно ли проверить сразу 32 начала окна?

SIMD (single instruction, multiple data) — одна инструкция обрабатывает вектор из многих байтов: 32 сравнения за раз.

AVX2 — конкретный набор таких инструкций x86 с 256-битными регистрами (32 байта); SSE2 — 16 байт, AVX-512 — 64, NEON на ARM — 16. Наша модель написана прямо на AVX2; библиотеки пишут код над абстрактным вектором.

Идея фильтра: выбрать в образце два байта и векторно проверить их сразу во многих позициях. Кто прошёл — проверяется точно, `memcmp`.

Денис Ярошевский, *SIMD Substring in a String*, C++Online 2025: запись; Wojciech Muła, SIMD-friendly algorithms for substring searching.

Два байта — необходимое условие, маска — кандидаты

Позиции a, b в образце: вхождение в s возможно, только если $T[s+a] = P[a]$ и $T[s+b] = P[b]$.

	0	1	2	3	4	5	6	7
$T[s]$	a	b	a	c	a	b	a	d
$T[s+1]$	b	a	c	a	b	a	d	x
$= a$	1	0	1	0	1	0	1	0
$= b$	1	0	0	0	1	0	0	0
AND	1	0	0	0	1	0	0	0

Для образца ab ($a = 0$, $b = 1$) остаются начала 0 и 4; для длинного образца это ещё кандидаты. Две загрузки, два сравнения, одно И — маска кандидатов; по её единицам — точная проверка.

Наша AVX2-модель: 32 начала за итерацию

```
35 __attribute__((target("avx2")))
36 int pair_filter(unsigned char* p,int m,unsigned char* t,int
       n) {
37     if(m==0) return n+1;
38     if(m>n) return 0;
39     int i=0,count=0,windows=n-m+1;
40     auto a=_mm256_set1_epi8(p[0]), b=_mm256_set1_epi8(p[m-1]);
41     for(; i+32<=windows; i+=32) {
42         auto x=_mm256_loadu_si256((const __m256i*)(t+i));
43         auto y=_mm256_loadu_si256((const __m256i*)(t+i+m-1));
44         unsigned mask=_mm256_movemask_epi8(_mm256_and_si256(
45             _mm256_cmpeq_epi8(x,a),_mm256_cmpeq_epi8(y,b)));
46         while(mask) {
47             int offset=__builtin_ctz(mask);
48             count+=std::memcmp(t+i+offset,p,m)==0;
49             mask&=mask-1;
50         }
51     }
52     for(;i<windows;++i) count+=std::memcmp(t+i,p,m)==0;
53     return count;
54 }
```

- Строка 40: 32 копии первого и последнего байта.
- Строки 42–45: две загрузки со сдвигом $m-1$, два сравнения, И, `movemask` — 32 бита кандидатов.
- Строки 46–50: `ctz` — младшая единица; `memcmp` проверяет окно; `mask&=mask-1` снимает бит.
- Строка 52: хвост — скалярно.

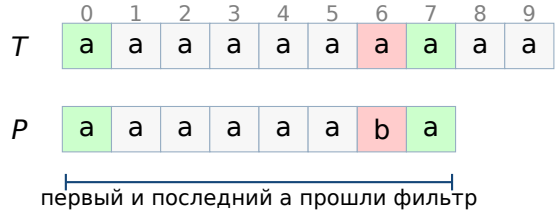
Учебная реализация курса, подготовлена с помощью ИИ; demo/search_bench.cpp:35–54. `target("avx2")`: вызывающий код проверяет поддержку AVX2.

Вопрос залу

Текст — миллион букв a. Образец — $a^{126}ba$. Что сделает фильтр по первому и последнему байту?

Вхождений нет: в тексте нет b. Сколько кандидатов пройдёт фильтр?

Ответ: фильтр пропускает всех — и стоит дороже, чем ничего



Оба выбранных байта — a: все 10^6 позиций — кандидаты. Каждую проверяет `memcmp`, который отвергает её на 127-м байте.

Наш замер на этом входе: **1,908 мс** с фильтром против **1,660 мс** у простого `memcmp` в каждом окне. Фильтр не отсёк никого и добавил свою цену — C_f при $q(1-f) \approx 0$.

../20261014/demo/search-comparison-results.csv, вход pair_trap; numbers.md, N09.

Что делает настоящая библиотека: Rust memchr

`memchr` — крейт, через который ищут подстроки стандартная библиотека Rust и ripgrep. Автор Andrew Gallant (BurntSushi).

Из README, раздел «Algorithms used»:

> *In all other cases, Two-Way is used. If possible, a prefilter based on the "Generic SIMD" algorithm … is used to find candidates quickly.* ***A dynamic heuristic is used to detect if the prefilter is ineffective, and if so, disables it.***

Точная проверка — Two-Way, как в CPython. Фильтр — пара байтов, как у нас, но пара выбирается по редкости байтов. И за фильтром *следят*: тот же счётчик, что `hits` в CPython.

Код над абстрактным вектором собирается под SSE2, AVX2 и NEON.

memchr 2.7.6, README; код маски: src/arch/generic/packedpair.rs:236–255 (в refs/).

Сравнение реализаций

Девять реализаций, четыре входа, один стенд

Как сравниваем скорость

- Один контракт для всех: **считаем все вхождения**, включая перекрытия.
- Подготовка, временные массивы и подсчёт входят во время: образец каждый раз новый.
- Семь серий, в каждой движки в случайном порядке; медиана на один проход. Текст уже в памяти.
- Чужой код как есть: SMART (КМП, BM, AG, Two-Way), AtCoder (Z), glibc `memmem`. Наши модели: наивный, `memcmp` в каждом окне, AVX2-пара.
- 32 439 проверок правильности перед замером: все движки дали одинаковые ответы.

CPU 1 · Ryzen AI MAX+ 395 · GCC 13.3 -O3 · glibc 2.39 · demo/run_comparison.py · numbers.md, N09.

Четыре входа — четыре разные нагрузки

Текст	Байт	Образец	Вхождений
Случайные 26 букв	1 048 576	`abracadabrax`	0
Английский словарь (370 105 слов)	4 234 910	`tion`	10 910
Только буква a	1 048 576	a^{128}	1 048 449
Только буква a	1 048 576	a^{126}`ba`	0

Словарь — тот же файл, что в лекции 4. Два последних входа — худшие случаи: совпадение везде и ловушка для фильтра по паре байтов.

Редкие совпадения: побеждает дешёвый фильтр

Медиана, мс (меньше — лучше)	26 букв, 0 вхождений	словарь, `tion`
Наивный посимвольный	0,630	3,191
`memcmp` в каждом окне	1,312	5,310
AtCoder Z: *P#T*	3,253	17,863
SMART КМП	1,667	7,179
SMART Бойер–Мур	0,374	4,703
SMART AG (+`memmove`)	0,584	7,137
SMART Two-Way	0,486	3,511
glibc `memmem`, в цикле	0,054	1,065
Наша AVX2-пара	**0,026**	**0,235**

Сравнивать внутри столбца. Наивный поиск здесь быстрее КМП: различие в первой букве, а у КМП больше работы на символ.

../20261014/demo/search-comparison-results.csv · 7 серий · numbers.md, N09.

Повторы: побеждает гарантия худшего случая

Медиана, мс (меньше — лучше)	$P = a^{128}$, 1 048 449 вхождений	$P = a^{126}ba$, 0 вхождений
Наивный посимвольный	26,956	26,354
`memcmp` в каждом окне	1,649	1,660
AtCoder Z: *P#T*	3,446	3,429
SMART КМП	1,433	1,436
SMART Бойер–Мур	26,795	1,847
SMART AG (+`memmove`)	9,813	4,725
SMART Two-Way	**1,033**	**0,411**
glibc `memmem`, в цикле	44,304	1,436
Наша AVX2-пара	1,877	1,908

Бойер–Мур на a^{128} — как наивный: после каждого совпадения сравнивает всё заново. AG сравнивает меньше, но двигает массив.

../20261014/demo/search-comparison-results.csv · 7 серий · numbers.md, N09.

За что платит каждая реализация

- **AVX2-пара:** 20 раз быстрее на редких совпадениях; на ловушке медленнее `memcmp` — фильтр не отсёк никого. Поэтому memchr его выключает.
- **Z через *P#T*:** 17,9 мс на словаре — два временных массива по `int` на байт, 32 MiB. Цена интерфейса, не алгоритма.
- **SMART AG:** линейное число сравнений, но `memmove` массива `skip` на каждом окне — хуже КМП.
- **glibc:** 44 мс — миллион вызовов с подготовкой в каждом; один поиск так не стоит.
- **Two-Way:** ни одного худшего случая и ни одной таблицы — поэтому он в glibc, CPython и Rust.

numbers.md, N09; memchr: отключение бесполезного фильтра.

Что выбрать для своей задачи

Задача	С чего начать
Первое вхождение, текст в памяти	библиотечный `find` / `memmem`: Two-Way + фильтр
Текст приходит потоком; все перекрытия	КМП: $O(n+m)$, память $O(m)$, состояние — одно число
Длинный образец, большой алфавит, редкие совпадения	Бойер–Мур или SIMD-фильтр; проверить замером
Совпадения каждой позиции с началом строки	Z-функция: ответ сразу для всех позиций
Много образцов сразу	Ахо–Корасик — следующий раздел

n — длина текста, m — образца. Если образец повторно используется, подготовку меряют отдельно: здесь она входила в каждый вызов.

Измерения N09 плюс доказанные свойства; это ориентиры, не рейтинг.

Ахо–Корасик

Тысячи образцов, один проход по тексту

А если образцов тысячи?

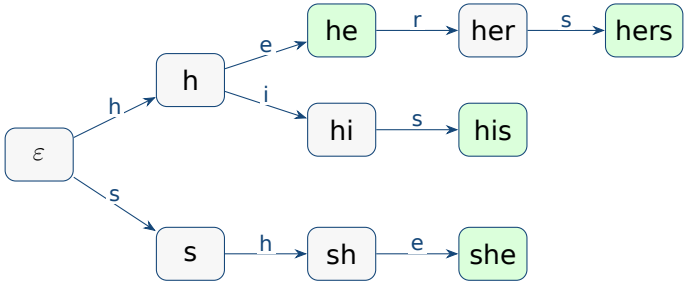
Антивирус, фильтр спама, `grep -F` со списком слов: один текст, много образцов. Запускать КМП для каждого — $O(n \cdot \text{число образцов})$.

Ахо и Корасик (1975): объединить все образцы в одно **trie** (лекция 4) и обобщить КМП: вместо одной границы — суффиксная ссылка в узле. Текст читается один раз.

Образцы: `he`, `she`, `his`, `hers`. Текст: `ushers`. Нужно найти все пары «позиция, номер образца».

Aho, Corasick, *Efficient string matching: an aid to bibliographic search*, CACM 1975.

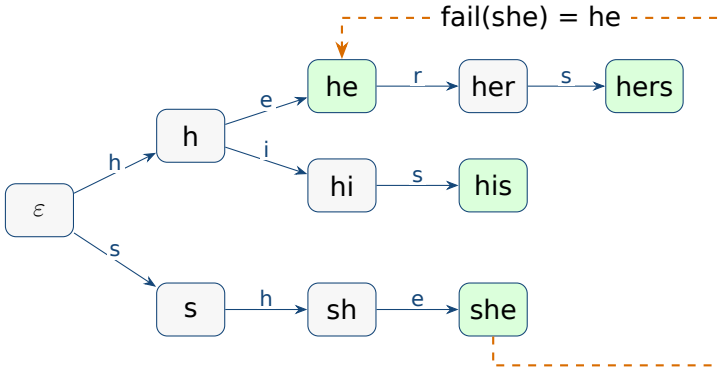
Trie объединяет общие префиксы



В узле — путь от корня; зелёные узлы заканчивают образец.

Состояние после чтения текста — самый длинный суффикс прочитанного, который является путём в trie. После ush это sh: предшествующая u уже не нужна.

Суффиксная ссылка: куда идти после несовпадения



$\mathrm{fail}(v)$ — самый длинный собственный суффикс пути v, который есть в trie.

Для she ссылка ведёт в he; для sh — в h; для he — в корень.

Это π из КМП: там граница была числом, здесь — узлом.

Прочитали ushe. Состояние — узел she. Что выдать?

Терминальный узел один — she. Но образцов четыре.

Ответ: выдать всё по цепочке ссылок

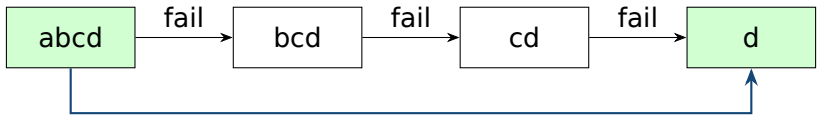
Прочли	Состояние	Выдача (позиция, образец)
u	корень	—
us	s	—
ush	sh	—
ushe	she	(1, she), (2, he)
usher	her	—
ushers	hers	(2, hers)

В состоянии she кончается и he: он — суффикс, то есть лежит на цепочке fail. Один флаг узла потерял бы его.

Шаг r: из she нет ребра r; $\mathrm{fail}(\mathrm{she}) = \mathrm{he}$, оттуда ребро r ведёт в her.

Выходная ссылка: прыгать сразу к следующему ответу

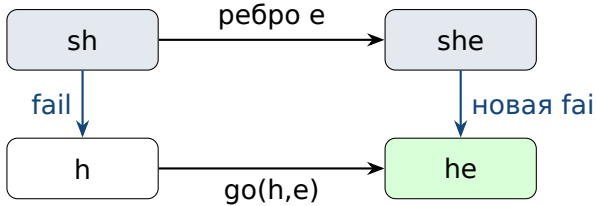
Другой пример: образцы abcd, bcde, cdef, d.



Цепочка fail от abcd: bcd → cd → d. Два промежуточных узла — не образцы; идти по ним на каждом символе дорого.

$\mathrm{exit}(v)$ — ближайший терминальный узел на цепочке fail. От abcd сразу к d. Тогда каждый шаг выдачи даёт ответ: $O(n + \mathrm{occ})$.

Почему ссылки строятся в ширину



Известно $\mathrm{fail}(\mathsf{sh}) = \mathsf{h}$. Из h по букве e попадаем в he. Значит, $\mathrm{fail}(\mathsf{she}) = \mathsf{he}$. В общем виде:

$$\mathrm{fail}(u) = \mathrm{go}(\mathrm{fail}(v), c) \quad \text{для ребра } v \xrightarrow{c} u.$$

Нужны готовые переходы более коротких путей — поэтому обход в ширину, по уровням. Для детей корня $\mathrm{fail} =$ корень отдельно.

Построение и поиск

```
while queue is not empty:
    v = queue.pop_front()
    for c in alphabet:
        if child[v][c] exists:
            u = child[v][c]
            go[v][c] = u
            f = go[fail[v]][c]
            fail[u] = f
            exit[u] = f if output[f] else exit[f]
            queue.push(u)
        else:
            go[v][c] = go[fail[v]][c]
```

```
v = root
for i = 0 .. n−1:
    v = go[v][T[i]]
    u = v
    while u != root:
        for id in output[u]:
            emit(i−length[id]+1, id)
        u = exit[u]
```

- Построение — BFS: строки 5–8 — ссылка через переход родителя, 9 — выходная ссылка; 11 — отсутствующее ребро заменяется готовым переходом: **полный автомат**, один переход на символ.
- Поиск: строка 3 — переход; 4–7 — выдача по цепочке exit.
- Цена: подготовка и память $O(V\sigma + L)$, поиск $O(n + \mathrm{occ})$.

Учебный псевдокод (начало — pseudo/ac_init.txt: корень и его дети). V — узлов, L — сумма длин образцов, σ — алфавит.

Настоящий код: Ахо–Корасик из KACTL, узел и BFS

```
47    queue<int> q;
48    for (q.push(0); !q.empty(); q.pop()) {
49      int n = q.front(), prev = N[n].back;
50      rep(i,0,alpha) {
51        int &ed = N[n].next[i], y = N[prev].next[i];
52        if (ed == -1) ed = y;
53        else {
54          N[ed].back = y;
55          (N[ed].end == -1 ? N[ed].end :
      backp[N[ed].start])
56            = N[y].end;
57          N[ed].nmatches += N[y].nmatches;
58          q.push(ed);
59        }
60      }
61    }
```

- Узел (строки 22–25): next[26] — переходы, back — fail, end — образец узла.
- Строка 52: y — переход из fail родителя; 53: пустое ребро заполняется им; 55: иначе это fail ребёнка.
- Строки 56–57: связывают выходы (наш exit).

kth-competitive-programming/kactl · CC0 content/strings/AhoCorasick.h:47–61

KACTL: поиск — один переход на символ

дополнительно

```
63  vi find(string word) {
64    int n = 0;
65    vi res; // ll count = 0;
66    for (char c : word) {
67      n = N[n].next[c - first];
68      res.push_back(N[n].end);
69      // count += N[n].nmatches;
70    }
71    return res;
72  }
73  vector<vi> findAll(vector<string>& pat, string word) {
74    vi r = find(word);
75    vector<vi> res(sz(word));
76    rep(i,0,sz(word)) {
77      int ind = r[i];
78      while (ind != -1) {
79        res[i - sz(pat[ind]) + 1].push_back(ind);
80        ind = backp[ind];
81      }
82    }
83    return res;
84  }
```

- Строка 67: весь поиск — `n = next[c]`. `find` даёт для позиции только *самый длинный* образец: ещё не все совпадения.
- Строки 76–81: `findAll` разворачивает цепочку `backp`: вложенные и повторные образцы; начало $= i - |\text{pat}| + 1$.

Задача из биологии

Сравнить два генома: как MUMmer свёл это к поиску подстрок

Задача из биологии: сравнить два генома

Чем отличаются два штамма одной бактерии?

Геном — для нас просто **строка над алфавитом из четырёх букв** A, C, G, T.

Организм	Длина генома, букв
Mycoplasma genitalium	580 074
Mycobacterium tuberculosis (возбудитель туберкулёза)	$\approx 4{,}4$ млн
человек	$\approx 3{,}1$ млрд

Два штамма туберкулёза: лабораторный H37Rv и клинический CDC1551 — по 4,4 млн букв. Биологи хотят знать, *где именно* они совпадают и где различаются.

Delcher et al., *Alignment of whole genomes*, NAR 1999; Marçais et al., *MUMmer4*, PLOS Comp Bio 2018. Полные ссылки — sources.pdf.

Что значит «сравнить»: выравнивание

Найти соответствие позиций двух строк, которое объясняет различия как

- замены одной буквы — **SNP** (single nucleotide polymorphism);
- вставки и удаления коротких участков;
- перестановки и перевороты больших кусков (на другой цепи ДНК).

A	a	g	c	t	-	a
B	-	g	c	t	t	a

Учебный пример: `agcta` и `gctta` — общие куски `gct` и `ta`, в каждой строке по вставке.

Зачем: эволюция штаммов, мутации устойчивости к лекарствам, проверка сборки генома.

Вопрос залу

Как сравнить две строки по 4,4 миллиона букв?

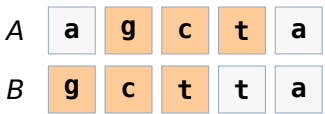
Классический ответ — динамическое программирование (редакционное расстояние): таблица $n \times m$, каждая клетка за $O(1)$.

Оцените число клеток для двух геномов туберкулёза. А для человека и шимпанзе (3,1 и 3,3 миллиарда)?

Ответ: $2 \cdot 10^{13}$ клеток — нельзя; ищем якоря

Идея Делчера и соавторов (1999), система **MUMmer**:

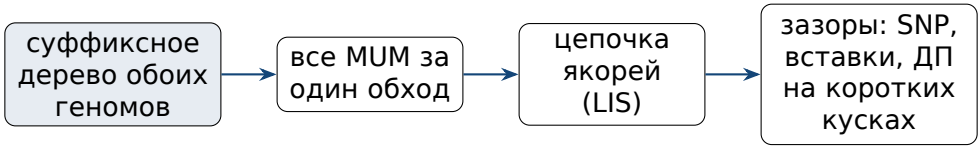
1. Найти **MUM** — maximal unique matches: подстроки, которые встречаются *ровно один раз* в каждом геноме и которые нельзя продлить ни влево, ни вправо.
2. Выбрать из них цепочку, идущую по порядку в обоих геномах (наибольшая возрастающая подпоследовательность).
3. Между соседними якорями остаются *короткие* зазоры: SNP, вставки, повторы. Их выравнивают ДП — таблицы уже маленькие.



MUM: `gct` и `ta`; зазоры: `a` слева, `t` справа

Delcher et al. 1999, разделы Methods: «MUM decomposition», «sorting the MUMs», «closing the gaps».

MUMmer, 1999: конвейер и настоящие числа



Пара геномов	размер	дерево	MUM + LIS	зазоры (ДП)
M. tuberculosis H37Rv / CDC1551	$2 \times 4{,}4$ млн	5 с	45 с	5 с
M. genitalium / *M. pneumoniae*	0,58 / 0,82 млн	6,5 с	0,02 с	116 с

DEC Alpha 4100, 1999 год. Память: не больше **37 байт на букву** генома. Дерево строится алгоритмом МакКрейта за линейное время.

Delcher et al. 1999, Results; по данным авторов, на их железе.

Что нужно от структуры данных

Опорный геном один, запросов много: штаммы, прочтения секвенатора, обе цепи ДНК.

- Построить индекс текста **один раз**, за $O(n)$.
- Отвечать на «где в тексте встречается P» за время, зависящее от m, а не от n.
- Находить *максимальные* общие куски двух текстов, а не только заданный образец.

MUMmer 3 (2004): дерево только опорного генома, запрос прогоняется по нему; 32-битное дерево — предел около 500 млн букв. MUMmer 4 (2018): вместо дерева **суффиксный массив** с 48-битными индексами; человек против шимпанзе ($3{,}09$ и $3{,}31$ млрд букв) — 3 ч 6 мин на 32 потоках, 66 ГБ. Сегодня — дерево; лекция 7 — массив и почему.

Marçais, Delcher, Phillippy, Coston, Salzberg, Zimin, *MUMmer4: A fast and versatile genome alignment system*, PLOS Comput Biol 14(1), 2018; Kurtz et al., Genome Biology 5:R12, 2004 (MUMmer 3).

Суффиксное дерево

Готовим текст, а не образец: все суффиксы в $O(n)$ узлов

Подготовить образец или подготовить текст?

Один текст, много разных запросов

Всё до сих пор готовило *образец*: таблицы КМП и Бойера–Мура, разрез Two-Way, автомат Ахо–Корасика. Текст читался заново для каждого запроса.

Что повторяется	Что готовим	Алгоритмы
Один образец, много текстов	образец	КМП, Бойер–Мур, Two-Way
Набор образцов, много текстов	общий автомат	Ахо–Корасик
Один текст, много образцов	**текст**	суффиксное дерево (сегодня), суффиксный массив (лекция 7)

Хочется ответа за $O(m + \mathrm{occ})$: длина образца плюс число найденных вхождений, и ни одного символа n.

Подстрока — это префикс суффикса

Начало	Суффикс $T\$$, $T =$ abaabaabacb
0	abaabaabacb$
1	baabaabacb$
2	aabaabacb$
3	abaabacb$
4	baabacb$
5	aabacb$
6	abacb$
7	bacb$
8	acb$
9	cb$
10	b$
11	$

P входит в T с позиции $s \iff P$ — префикс суффикса $T[s..)$.
abaab — префикс суффиксов 0 и 3. Все вхождения — все суффиксы с таким префиксом.
Положим все суффиксы в **trie** (лекция 4): поиск образца — спуск по trie за $O(m)$, вхождения — листья под достигнутой точкой.
Терминатор $ не встречается в T: ни один суффикс не является префиксом другого, каждый кончается в своём листе.

Вопрос залу

Сколько узлов может быть в trie всех суффиксов строки длины n?

Подсказка: каждый узел — различная подстрока. Сколько различных подстрок у $a^k b^k$?

Ответ: до $\Theta(n^2)$ — и как сжать до $O(n)$

Для $a^k b^k$ все k^2 строк $a^i b^j$ различны: узлов $\Theta(n^2)$.

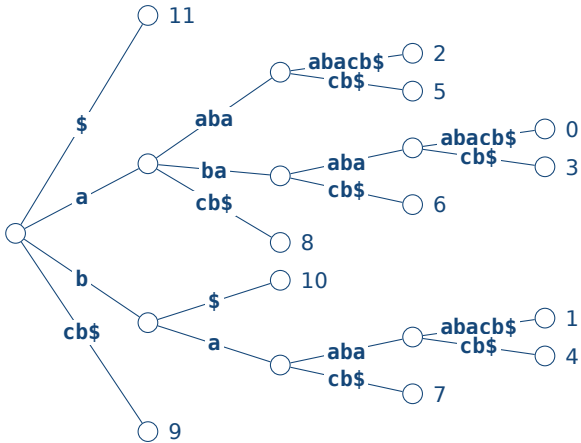
Но *ветвится* путь редко. Два изменения сразу:

1. Убрать узлы с одним ребёнком: цепочка без ветвления становится одним ребром с меткой-строкой.
2. Метку ребра хранить как **пару индексов** в T, не копируя буквы. Текст хранится один раз.

Суффиксное дерево T: сжатый trie всех суффиксов $T\$$; у внутреннего узла ≥ 2 ребёнка; листьев ровно $n+1$, в листе — начало суффикса.

Weiner 1973; Gusfield, *Algorithms on Strings, Trees, and Sequences*, гл. 5; лекция 06.11.2009.

Суффиксное дерево строки abaabaabacb$



Числа у листьев — начала суффиксов. Путь abaab кончается *внутри* ребра над листьями 0 и 3.

Размер: не больше $2n + 1$ узлов

Листьев $L = n + 1$ (по одному на суффикс $T\$$). У каждого внутреннего узла, кроме, может быть, корня, не меньше двух детей.

Дерево с L листьями, в котором каждый внутренний узел ветвится, имеет не больше $L - 1$ внутренних узлов:

$$E = I + L - 1, \qquad E \geq 2I \quad \Longrightarrow \quad I \leq L - 1$$

(рёбер на один меньше, чем узлов; у каждого внутреннего узла не меньше двух рёбер вниз).

Итого узлов $\leq 2n + 1$, рёбер $\leq 2n$. На ребро — два индекса, на лист — одно число, текст — n символов: **память $O(n)$ слов**, если переходы по первому символу хранятся без таблицы на весь алфавит.

Gusfield, лемма 5.2.1 в других обозначениях.

Поиск: $O(m + \mathrm{occ})$, и он может кончиться посреди ребра

Из корня: по первому символу P выбираем ребро, сравниваем символы метки с P, спускаемся.

- Символ не совпал — вхождений нет.
- P закончился — **успех**, даже посреди ребра: все листья под этой точкой — вхождения.

Прочитано не больше m символов; листьев под точкой ровно occ, их обход — $O(\mathrm{occ})$.

Если заранее записать в каждом узле число листьев под ним, вопрос «сколько раз?» стоит $O(m)$. Пример: abaab в дереве выше — путь a·ba·ab… кончается внутри ребра aba, под ним листья 0 и 3.

Наивное построение и почему оно квадратично

Вставлять суффиксы по одному, каждый — от корня: идти по совпадающим меткам, при различии разрезать ребро и добавить лист.

- Результат правильный и компактный.
- Но суффикс j читается с начала: на a^n суффикс j проходит $n - j$ символов — всего $\Theta(n^2)$.

Год	Кто	Как растёт дерево
1973	Weiner	справа налево; «алгоритм 1973 года», по словам Кнута
1976	McCreight	суффиксы от длинного к короткому; это в MUMmer 1999
1995	Ukkonen	**слева направо, символ за символом** — разбираем его

Apostolico, Crochemore, Farach-Colton, Galil, Muthukrishnan, *40 Years of Suffix Trees*, CACM 59(4), 2016;

Ukkonen, Algorithmica 1995.

1973 год: гипотеза Кнута прожила недолго

На заре «стрингологии» Дональд Кнут высказал гипотезу: найти **самую длинную общую подстроку** двух строк суммарной длины n нельзя быстрее, чем за $\Omega(n \log n)$. Алгоритм за $O(n \log n)$ уже был (Карп, Миллер, Розенберг, 1972).

В 1973 году Питер Вайнер показал линейное решение для фиксированного алфавита — и оно оказалось *побочным продуктом*: Вайнер строил структуру для другой задачи — «узнавать любую подстроку файла, не перечисляя их все», то есть инвертированный индекс текста.

Он назвал её **bi-tree**; среди применений в статье — поиск образца в заранее обработанном файле за время, линейное по длине образца. Это ровно наш $O(m + \mathrm{occ})$.

По рукописи Вогана Пратта, Кнут назвал это **«алгоритмом 1973 года»**.

Apostolico et al., *40 Years of Suffix Trees*, CACM 2016, §2; Weiner, SWAT 1973. «Алгоритм года» — по Giegerich, Kurtz 1997, §6, со ссылкой на рукопись Pratt 1973.

Сорок лет одной структуры

Год	Кто	Что
1973	Weiner	bi-tree; текст читается справа налево; опровергнута гипотеза Кнута
1973	Pratt	заметки «Improvements and applications for the Weiner repetition finder» (не опубликованы)
1976	McCreight	построение слева направо, экономная память; **имя «suffix tree»**
1995	Ukkonen	**онлайн**-построение, символ за символом — через два десятилетия
1997	Farach	линейное построение для любого целочисленного алфавита
1999	Delcher et al.	MUMmer: дерево двух геномов, построение по McCreight
2013	CPM	сессия «40 Years of Suffix Trees»: Weiner, Pratt, McCreight, Ziv на одной сцене

До 1997 года «линейно» значит «при фиксированном алфавите». Память дерева оставалась «головной болью» — отсюда массивы, лекция 7.

Apostolico et al., CACM 2016, §2 и список литературы; записи сессии CPM 2013: vimeo.com/channels/574784.

Алгоритм Укконена

Построить дерево за $O(n)$, читая текст слева направо

Как не начинать каждый суффикс от корня?

Идея: держать дерево *всех суффиксов уже прочитанного префикса* и при каждом новом символе **дописывать его ко всем суффиксам сразу**.

Прочитано $S[0..i)$. Пришёл символ $S[i]$. Каждый суффикс $S[j..i)$ должен превратиться в $S[j..i)\,S[i]$; плюс появляется новый пустой суффикс $S[i..i+1)$.

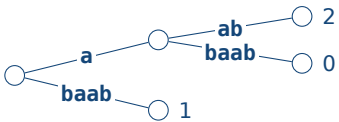
$$\mathsf{aba} \longrightarrow \mathsf{abaa}: \qquad \mathsf{aba} \to \mathsf{abaa},\ \mathsf{ba} \to \mathsf{baa},\ \mathsf{a} \to \mathsf{aa},\ \varepsilon \to \mathsf{a}.$$

Пример дня: $S = \mathsf{abaab\$}$ — образец лекции 5 с терминатором. Шесть фаз.

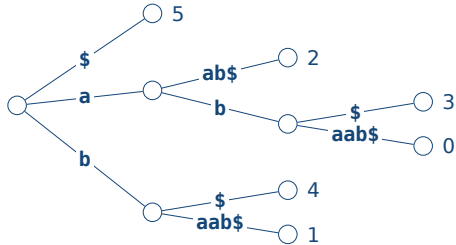
Ukkonen, *On-line construction of suffix trees*, Algorithmica 14, 1995; Gusfield, гл. 6; лекция 06.11.2009.

Неявное суффиксное дерево

Для abaab (без терминатора):



Для abaab$:



Без терминатора суффиксы ab, b — префиксы других суффиксов: они кончаются *внутри рёбер*, своих листьев у них нет. Узлов с одним ребёнком тоже нет.

Неявное суффиксное дерево $S[0..i)$ — дерево всех его суффиксов без терминатора. Алгоритм строит неявные деревья префиксов $S[0..1)$, $S[0..2)$, ...; терминатор в конце делает дерево настоящим.

Фазы и продолжения

Фаза $i+1$ превращает дерево $S[0..i)$ в дерево $S[0..i+1)$. Внутри неё **продолжение** j для $j = 0, 1, \ldots, i$: найти в дереве конец пути $\beta = S[j..i)$ и продолжить его символом $S[i]$.

```
1 build(S):                          # S ends
      with a unique terminator
2     tree = empty
3     for i = 0 .. n−1:              # phase
      i+1: append S[i]
4         for j = 0 .. i:            #
      extension j: suffix S[j..i)
5             find the end of path S[j..i)
      from the root
6             apply rule 1, 2 or 3 with
      symbol S[i]
```

- Строка 3: n фаз; строка 4: в фазе $i+1$ продолжения $j = 0..i$, суффиксы от длинного к короткому.
- Строки 5–6: найти конец β спуском от корня, применить одно из трёх правил.
- Как написано — $O(n^3)$. Дальше убираем два множителя.

Gusfield, §6.1.1; лекция 06.11.2009, «Общий вид алгоритма».

Три правила продолжения

Нашли конец пути $\beta = S[j..i)$. Нужно, чтобы $\beta S[i]$ тоже был в дереве.

	Где кончается β	Что делать
1	в листе	дописать $S[i]$ к метке листового ребра
2	в узле или внутри ребра, и дальше *нет* пути по $S[i]$	создать новый лист с номером j; если β кончилась внутри ребра — сначала разрезать его новым узлом
3	дальше *уже есть* путь по $S[i]$	ничего: $\beta S[i]$ уже в дереве

Правило 2 — единственное, которое меняет форму дерева.
Правило 3 — место, где экономят.

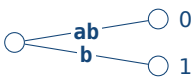
Лекция 06.11.2009, «Правила продолжения суффиксов»; Gusfield, §6.1.1.

Фазы 1–3: a, ab, aba

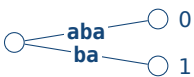
фаза 1, a



фаза 2, ab



фаза 3, aba



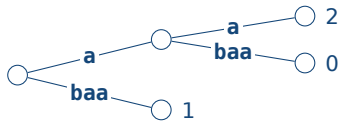
Фаза	$j = 0$	$j = 1$	$j = 2$
1 (a)	правило 2: лист 0		
2 (ab)	правило 1: a→ab	правило 2: лист 1	
3 (aba)	правило 1	правило 1	правило 3: путь a уже есть

Фаза 4: к aba добавляем a. Какие правила сработают для $j = 0, 1, 2, 3$?



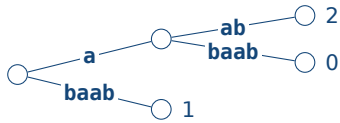
Суффиксы, которые надо продолжить: aba, ba, a, ε.

Ответ: фаза 4 — первый разрез



j	β	где кончается	правило
0	aba	лист 0	1: aba→abaa
1	ba	лист 1	1: ba→baa
2	a	внутри ребра aba, дальше b≠a	**2**: разрезать после a, новый лист 2
3	ε	корень, ребро по a есть	3

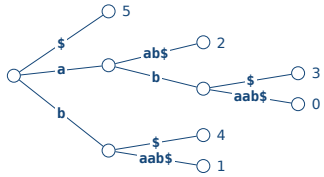
Фаза 5: abaab — правило 3 закрывает фазу



j	β	где кончается	правило
0, 1, 2	abaa, baa, aa	листья 0, 1, 2	1: к каждой метке дописан b
3	a	узел a; из него *есть* ребро по b	**3**: ab уже в дереве
4	ε	корень; ребро по b есть	3

Форма дерева не изменилась: выросли только метки листьев. Суффиксы ab и b живут внутри рёбер.

Фаза 6: терминатор — только правила 1 и 2



j	β	где кончается	правило
0, 1, 2		листья	1
3	ab	внутри ребра baab из узла a, дальше a≠$	**2**: узел ab, лист 3
4	b	внутри ребра baab из корня, дальше a≠$	**2**: узел b, лист 4
5	ε	корень, ребра по $ нет	**2**: лист 5

$ ни с чем не совпадает, поэтому правило 3 не сработает ни разу: каждый суффикс получает лист. Это настоящее суффиксное дерево.

Сколько это стоит и что лишнее

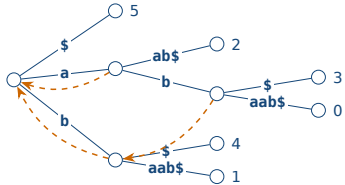
Как написано: n фаз $\times$ до n продолжений $\times$ спуск от корня до n символов — $O(n^3)$.

Три наблюдения из трассы:

1. Конец пути $S[j+1..i)$ очень близко к концу пути $S[j..i)$: это та же строка без первого символа. **Не спускаться от корня** — **суффиксные ссылки**.
2. Если для j сработало правило 3, для всех $j' > j$ тоже сработает. **Заканчивать фазу** при первом правиле 3.
3. Лист, однажды созданный, в каждой следующей фазе только растёт на один символ. **Не трогать листья вовсе**.

Каждое наблюдение убирает по множителю; вместе дают $O(n)$.

Суффиксная ссылка: убрать первый символ пути



Ссылка узла с путевой меткой $x\alpha$ ведёт в узел α: ab$\to$b, b$\to$корень, a$\to$корень.

Теорема. Если в продолжении j правило 2 создаёт узел $x\alpha$, то узел α либо уже есть, либо будет создан в продолжении $j+1$ той же фазы.

Почему: разрез случился из-за пути $x\alpha c$, $c \neq S[i]$; значит, есть путь αc. Если из конца α есть и другой путь — узел уже существует; иначе он возникнет, когда продолжение $j+1$ добавит $\alpha S[i]$.

Лекция 06.11.2009, «Создание суффиксных связей»; Gusfield, лемма 6.1.1.

Как ссылка ускоряет продолжение $j+1$

Продолжение j кончилось в точке пути $S[j..i)$. Пусть v — ближайший явный узел над ней, γ — остаток пути от v до точки.

1. Подняться в v (не выше одного ребра).
2. Перейти по ссылке $v \to s(v)$: путь $s(v)$ — это путь v без первого символа.
3. Спуститься по γ от $s(v)$: это и есть конец пути $S[j+1..i)$.

Прыжок по счётчику: спускаясь по γ, не сравнивать символы. γ заведомо есть в дереве; на каждом ребре смотрим только *первый* символ и *длину* ребра: если ребро короче остатка — прыгаем в его конец целиком.

Цена спуска — число пройденных рёбер, а не символов.

Лекция 06.11.2009, «Прыжок по счётчику»; Gusfield, §6.1.3, skip/count trick.

Правило 3 завершает фазу; листья остаются листьями

Правило 3 завершает фазу. Если путь $S[j..i]$ уже есть, то есть и его суффиксы $S[j+1..i]$, $S[j+2..i]$, …Для всех следующих j сработает правило 3: фаза заканчивается при первом правиле 3.

Листья остаются листьями. Созданный лист в каждой следующей фазе получает правило 1. Пусть правая граница всех листовых рёбер — **одна переменная** e; в начале фазы $i+1$ присваиваем $e \leftarrow i+1$, и все продолжения по правилу 1 выполнены за $O(1)$.



метка $= S[1..e)$; растёт e — растёт метка

Лекция 06.11.2009, «Третье правило завершает фазу», «Листья остаются листьями»; Gusfield, §6.1.4–6.1.5.

Алгоритм одной фазы

```
1  phase(i+1):                    # tree of S[0..i) ->
       tree of S[0..i+1)
2      e = i+1                    # rule 1 for every leaf
       at once
3      for j = j_i + 1 .. i:      # skip extensions that
       are leaves already
4          go to the end of S[j..i):
5              from the node where extension j-1 ended,
       take its
6              suffix link and walk down by edge lengths
       (skip/count)
7          if a path by S[i] continues from there:      #
       rule 3
8              j_{i+1} = j - 1; return                   #
       phase is over
9          create leaf j (split the edge if needed)     #
       rule 2
10         if a node was created: set its suffix link
11             at the next extension
12     j_{i+1} = i
```

- j_i — последний лист к концу фазы i; продолжения $0..j_i$ — правило 1, строка 2.
- Строки 3–6: явные продолжения — по ссылке и прыжками.
- Строки 7–8: первое правило 3 закрывает фазу.
- Строки 9–11: правило 2; ссылка нового узла — в следующем продолжении.

j_i не убывает: ставшее листом листом и остаётся.

Откуда $O(n)$

- Явных продолжений за всё время $\leq 2n$: в фазе их $j^* - j_i$; $j_{i+1} \geq j^* - 1$, так что все, кроме одного на фазу, сдвигают j вперёд, а $j \leq n$.
- Одно явное продолжение: подняться на ≤ 1 ребро, ссылка, спуск прыжками, $O(1)$ работы в конце.
- Спуск прыжками: считаем *глубину в узлах* текущей точки. Подъём и ссылка уменьшают её не больше чем на 2 (ссылка из узла глубины d ведёт в узел глубины $\geq d - 1$); каждый прыжок увеличивает на 1. Уменьшений $\leq 2 \cdot 2n$, значит, прыжков $O(n)$.

Теорема. Алгоритм Укконена строит суффиксное дерево $S\$$ и все суффиксные ссылки за $O(n)$ при алфавите постоянного размера. Для алфавита σ — $O(n \log \sigma)$ или $O(n)$ с таблицами на σ в каждом узле.

Gusfield, лемма 6.1.2 и теорема 6.1.1; Ukkonen 1995, теорема 2; лекция 06.11.2009, «Время выполнения алгоритма».

Откуда мы знаем, что это работает

- **Доказательство** — про алгоритм: правила дают верное дерево каждого префикса, приёмы не меняют результат.
- **Наша модель** (tools/draw_tree.py) выполняет правила буквально; сравнена с наивной вставкой суффиксов на **5325 строках** (двоичные до длины 10, троичные до 7): расхождений нет. Рисунки — из неё.
- **Настоящий код:** cceh/suffix-tree повторяет статью Укконена с её именами (`canonize`, `test_and_split`) — следующий кадр.

Настоящий код: прыжок по счётчику в библиотеке

```
120         # follow transitions starting from s until we
    get too far down the tree,
121         # then return the transition before the one
    that went too far
122         s_prime, _S_prime, k_prime, p_prime =
    self.transition(s, start)
123
124         while p_prime - k_prime <= end - start:
125             # while the found transition ends higher
    on the tree, see if the
126             # next transition would lead us too far
    down the tree
127             start += p_prime - k_prime
128             assert isinstance(s_prime, Internal)
129             s = s_prime  # type: ignore
130
131             if end - start > 0:
132                 s_prime, _S_prime, k_prime, p_prime =
    self.transition(s, start)
```

- canonize из статьи Укконена: спуститься по остатку до ближайшего узла. `transition` — ребро по первому символу остатка.
- Строка 124: пока ребро не длиннее остатка; 127: сдвинуть начало остатка на длину ребра, *не читая символов*. Это прыжок по счётчику.

cceh/suffix-tree · b3040d8 · GPL-3.0 src/suffix_tree/ukkonen.py:120–133 (копия в 2026/research/suffix-indexes/refs)

Что умеет дерево

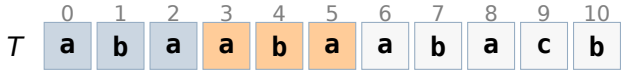
Повторы, общая подстрока двух текстов, MUM

Вхождения, их число, самый длинный повтор

После спуска по P:

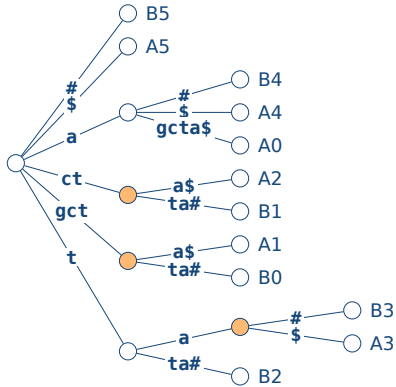
- все начала — листья под точкой: $O(m + \mathrm{occ})$;
- число вхождений — заранее посчитанное число листьев в поддереве: $O(m)$;
- самый длинный повторяющийся кусок текста — самый глубокий (по *длине пути*) внутренний узел: $O(n)$ один обход.

У abaabaabacb это abaaba, начала 0 и 3 — вхождения перекрываются.



Обобщённое дерево: два текста в одном дереве

Строки $A = \texttt{agcta}$ и $B = \texttt{gctta}$ — наши «геномы». Разные терминаторы: $A\$$ и $B\#$. Вставляем все суффиксы обеих. В листе — номер строки и начало.



Вопрос залу

Какой узел этого дерева — наибольшая общая подстрока *A* и *B*?

И второй: какие узлы — MUM, то есть куски, которые встречаются ровно один раз в *A* и ровно один раз в *B* и которые нельзя продлить ни влево, ни вправо?

Ответ: общая подстрока и MUM — это узлы

Наибольшая общая подстрока. Пометим узлы: есть ли под ними листья A и листья B. Самый глубокий узел с обеими пометками — `gct`, длина 3. Один обход, $O(|A| + |B|)$.

MUM — узел с *ровно двумя* листьями-детьми, по одному из каждой строки: по одному вхождению в каждой; пути расходятся — вправо не продлить; осталось проверить символы *слева* от вхождений.

Кандидат	в A	в B	слева
`gct`	1	0	a / начало — **MUM**
`ta`	3	3	c / t — **MUM**
`ct`	2	1	g / g — продлевается: не MUM

Все MUM — за один обход дерева. Это шаг 1 конвейера MUMmer, и он линеен.

Delcher et al. 1999, «MUM decomposition»; Gusfield, §7.4 (LCS), §7.12.

MUMmer целиком, ещё раз

1. Обобщённое дерево двух геномов (1999) — или дерево опорного генома и прогон запроса по нему (с 2004).
2. Все MUM — один обход дерева, $O(n)$.
3. Цепочка MUM, возрастающая в обоих геномах, — вариант наибольшей возрастающей подпоследовательности, $O(k \log k)$ по числу MUM.
4. Зазоры между соседними якорями коротки: SNP — зазор в одну букву; остальное — динамическое программирование на маленьких таблицах.

Квадратичная задача свелась к линейной структуре и небольшому ДП. Цена — память дерева: 37 байт на букву в 1999 году. Следующая лекция: как оставить от дерева два массива и чем за это платят.

Delcher et al. 1999; Marçais et al. 2018.

Статистика совпадений: длина совпадения с каждой позиции

дополнительно

Для запроса Q и текста T: $MS[i]$ — длина самого длинного префикса $Q[i..)$, который встречается в T.

i	0	1	2	3	4	5	6	7	8
$Q[i]$	b	a	a	b	a	x	a	b	a
$MS[i]$	5	4	3	2	1	0	3	2	1

$T =$ abaabaabacb. Наивно — $O(|Q|^2)$ спусков. С суффиксными ссылками — один проход: $MS[i+1] \geq MS[i] - 1$, и к концу найденного пути переходим ссылкой, как в Укконене, не возвращаясь к корню.

Так MUMmer 3 прогоняет запрос по дереву опорного генома без обобщённого дерева.

Gusfield, §7.8; таблица посчитана demo/matching_statistics.py.

Что уносим

- **Когда текст один, готовят текст.** Суффиксное дерево — все суффиксы в $O(n)$ узлов; поиск за $O(m + \mathrm{occ})$.
- **Укконен строит его за $O(n)$** тремя приёмами: суффиксные ссылки с прыжком по счётчику, правило 3 закрывает фазу, листья растут сами.
- **Повторы, общая подстрока, MUM — это узлы дерева.** Задача из биологии, которая не решалась динамическим программированием, свелась к одному обходу.
- **Цена — память.** 37 байт на букву в 1999 году; на следующей лекции — суффиксный массив и LCP, те же ответы в двух плотных массивах, и почему MUMmer4 выбрал их.

Спасибо за внимание

https://github.com/alk-2000/da-lectures